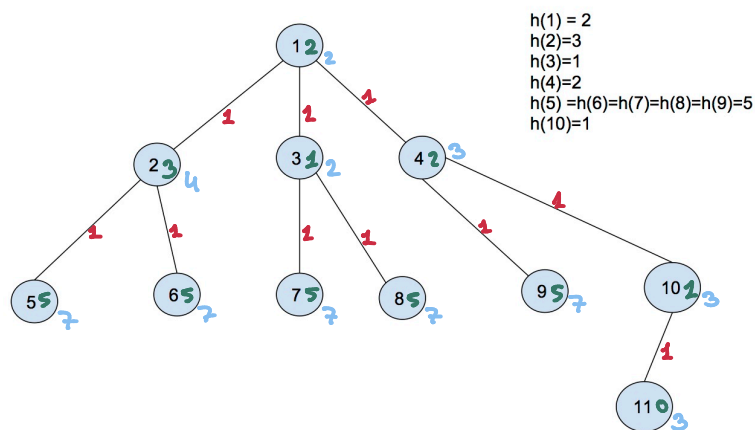


1º Teste de Inteligência Artificial  
16/4/2024- 10-12 horas

**Grupo 1** Considere que a árvore da figura 1 corresponde ao espaço de estados de um problema. O nó 1 é o estado inicial e o nó 11 é a solução do problema.



Considere que o custo de ir de um nó para outro sucessor é sempre 1, e considere a estimativa  $h(n)$  na figura 1.

1. Indique, justificando se a heurística,  $h$ , é admissível para este problema.
2. Para cada um dos algoritmos diga quais são os nós visitados indicando a ordem por que são visitados.
  - (a) Com a pesquisa em largura
  - (b) Com a pesquisa em profundidade

Grupo I

| 1. | Nó | $h(n)$ | Custo real mínimo | Admissível |
|----|----|--------|-------------------|------------|
|    | 1  | 2      | 3                 | ✓          |
|    | 2  | 3      | 4                 | ✓          |
|    | 3  | 1      | 4                 | ✓          |
|    | 4  | 2      | 2                 | ✓          |
|    | 5  | 5      | 5                 | ✓          |
|    | 6  | 5      | 5                 | ✓          |
|    | 7  | 5      | 5                 | ✓          |
|    | 8  | 5      | 5                 | ✓          |
|    | 9  | 5      | 3                 | ✗          |
|    | 10 | 1      | 1                 | ✓          |

A heurística não é admissível porque no nó 9 a estimativa  $h(n)$  superestima o custo real mínimo.

2. a) 1 234 5678910 //

b) 1256 378 49 10 //

c) 1 1234 12563784910 125637849 10 //

d) 1 3 4 10 // 2 5 6 7 8 9

e) 1 3 4 10 // 2 5 6 7 8 9

- (c) Com a pesquisa em profundidade iterativa.
- (d) Com o algoritmo ansioso (greedy).
- (e) Com o algoritmo A\*.

**Grupo 2** Considere o seguinte problema: Nas Salinas do Samouco o sr. Silva vende cristais de Sal aos visitantes, como iniciou o negócio há pouco tempo não tem balança e usa duas caixas de madeira, uma, a caixa 1 têm capacidade para 500 gramas e a outra, a caixa 2, tem capacidade para 200 gramas. O Sr. Silva obtêm as quantidades pedidas com as seguintes operações sobre as caixas: encher uma até à capacidade total; despejar totalmente uma caixa no saco de sal, despejar totalmente uma caixa na outra até que a outra esteja totalmente cheia ou que a que está a ser despejada esteja vazia; ou despejar uma caixa no saco do cliente.

Por exemplo, para vender 100 gramas de cristais de Sal o Sr. Silva faz as seguintes acções: enche a caixa 2 e a seguir despeja-a na caixa 1; enche a caixa 2, outra vez, e despeja-a na caixa 1; volta a encher a caixa 2 e despeja-a na caixa 1 até a caixa 1 estar cheia; fica com os 100 gramas na caixa 2 e despeja-a no saco do cliente.

Considere que no estado inicial o Sr Silva tem as caixas vazias e no estado final o cliente tem 300 gramas no saco.

1. Para representar este problema como um problema de pesquisa no espaço de estados, indique a informação que tem de representar num estado;
2. Represente o estado inicial (fig. acima) e o estado final.
3. Indique todas as ações que o agente pode realizar num estado e defina o operador de transição de estado em prolog.
4. Indique a solução óptima deste problema para o estado em que tem 300 gramas no saco e indique a profundidade a que está a sua solução na árvore de pesquisa.
5. Proponha uma heurística admissível para este problema, e implemente-a em Prolog.



## Grupo II

1. • Sal na caixa 1

• Sal na caixa 2

• Sal no saco do cliente

• Capacidades de  $c_1$  e  $c_2$

estado((  $c_1$ ,  $c_2$ ,  $sc$  ))

capacidade(  $c_1$ , 500)

capacidade(  $c_2$ , 200)

2. estado\_inicial(( 0, 0, 0))

estado\_final(( -, -, 300))

3. • Encher  $c_1$

op( estado\_atual, operador, estado\_seguinte, custo)

op((  $c_1$ ,  $c_2$ ,  $sc$  ), e(  $c_1$  ), (  $NC_1$ ,  $c_2$ ,  $sc$  ), 1) :- capacidade(  $c_1$ ,  $NC_1$  ),

• Encher  $c_2$

$c_1 \neq NC_1$ .

op((  $c_1$ ,  $c_2$ ,  $sc$  ), e(  $c_2$  ), (  $c_1$ ,  $NC_2$ ,  $sc$  ), 1) :- capacidade(  $c_2$ ,  $NC_2$  ),

• Despejar  $c_1$

$c_2 \neq NC_2$ .

op((  $c_1$ ,  $c_2$ ,  $sc$  ), d(  $c_1$  ), ( 0,  $c_2$ ,  $sc$  ), 1) :-  $c_1 \neq 0$ .

• Despejar  $c_2$

op((  $c_1$ ,  $c_2$ ,  $sc$  ), d(  $c_2$  ), (  $c_1$ , 0,  $sc$  ), 1) :-  $c_2 \neq 0$ .

• Despejar  $c_1$  em  $c_2$

max(  $A$ ,  $A$ ,  $B$  ) :-  $A > B$ , !.

max(  $A$ , -,  $A$  ).

min(  $A$ ,  $A$ ,  $B$  ) :-  $A < B$ , !.

min(  $A$ , -,  $A$  ).

op((  $c_1$ ,  $c_2$ ,  $sc$  ), d(  $c_1$ ,  $c_2$  ), (  $NC_1$ ,  $NC_2$ ,  $sc$  ), 1) :- capacidade(  $c_2$ ,  $Cap_2$  ),

Soma is  $c_1 + c_2$ ,

min( Soma,  $Cap_2$ ,  $NC_2$  ),

Sobra is  $Soma - NC_2$ ,

max( Sobra, 0,  $NC_1$  ),

(  $c_1 \neq NC_1$  ;  $c_2 \neq NC_2$  ).

• Despejar  $c_2$  em  $c_1$

op((  $c_1$ ,  $c_2$ ,  $sc$  ), d(  $c_2$ ,  $c_1$  ), (  $NC_1$ ,  $NC_2$ ,  $sc$  ), 1) :- capacidade(  $c_1$ ,  $Cap_1$  ),

Soma is  $c_1 + c_2$ ,

min( Soma,  $Cap_1$ ,  $NC_1$  ),

Sobra is  $Soma - NC_1$ ,

max( Sobra, 0,  $NC_2$  ),

(  $c_1 \neq NC_1$  ;  $c_2 \neq NC_2$  ).



- Despejar  $c_1$  totalmente em  $S$

$op((c_1, c_2, sc), d(c_1, saco), (0, c_2, NSC), 1) :- NSC \text{ is } sc + c_1,$   
 $c_1 > 0.$

- Despejar  $c_2$  totalmente em  $S$

$op((c_1, c_2, sc), d(c_2, saco), (c_1, 0, NSC), 1) :- NSC \text{ is } sc + c_2,$   
 $c_2 > 0.$

4.  $e(c_1) \rightarrow d(c_1, c_2) \rightarrow d(c_1, saco)$   
 $(500, 0, 0) \rightarrow (300, 200, 0) \rightarrow (0, 200, \underline{\underline{300}})$

A solução ótima está à profundidade 3 na árvore de pesquisa.

5.  $h(n) = \underbrace{|\underline{300} - \underline{sc}|}_{\substack{\text{OBJETIVO} \\ \text{valor no saco} \\ \text{do cliente}}}$

100

O menor despesa possível é de 100gr  
 E qualquer despesa realizada será múltiplo de 100

heurística (estado(-, -, sc), H) :- Dif is abs(300 - sc),

H is Dif // 100.

H is ceil(Dif / 100).

**Grupo 3** - Considere o problema de colocar  $2Xn$  peças num tabuleiro de  $n \times n$  de forma a que não existam mais de 4 peças em cada linha ou coluna (horizontal ou vertical).

|   |   |   |   |
|---|---|---|---|
| X | X |   |   |
|   | X |   |   |
| X |   |   | X |
| X |   | X | X |

Este problema pode ser representado e resolvido como um problema de satisfação de restrições (CSP).

1. Proponha uma representação em Prolog para os estados do problema. Indique o que representam as variáveis e qual o seu domínio.  
Exemplifique representando o estado inicial para  $n=4$ , e desenhe a árvore do espaço de estados até ao nível 3.
2. Defina o predicado *sucessor*( $E_i, E_s$ ) que sucede para todo  $E_i$ ,  $E_s$  em que  $E_s$  é um sucessor de  $E_i$ .
3. Represente em Prolog as restrições deste problema. E defina o predicado *verifica restrições*(*Estado*) que sucede quando um estado verifica todas as restrições.
4. Indique, justificando, se as seguintes estratégias podem cortar o espaço de pesquisa do problema:
  - a) Forward checking;
  - b) Escolher a variável mais restringida;
  - c) Escolher a variável menos restringida;



### Grupo III

1. Variáveis - 16 variáveis para posição

Domínio - 0 a 1 que representam a existência de um X, ou não.

0 = não existe X

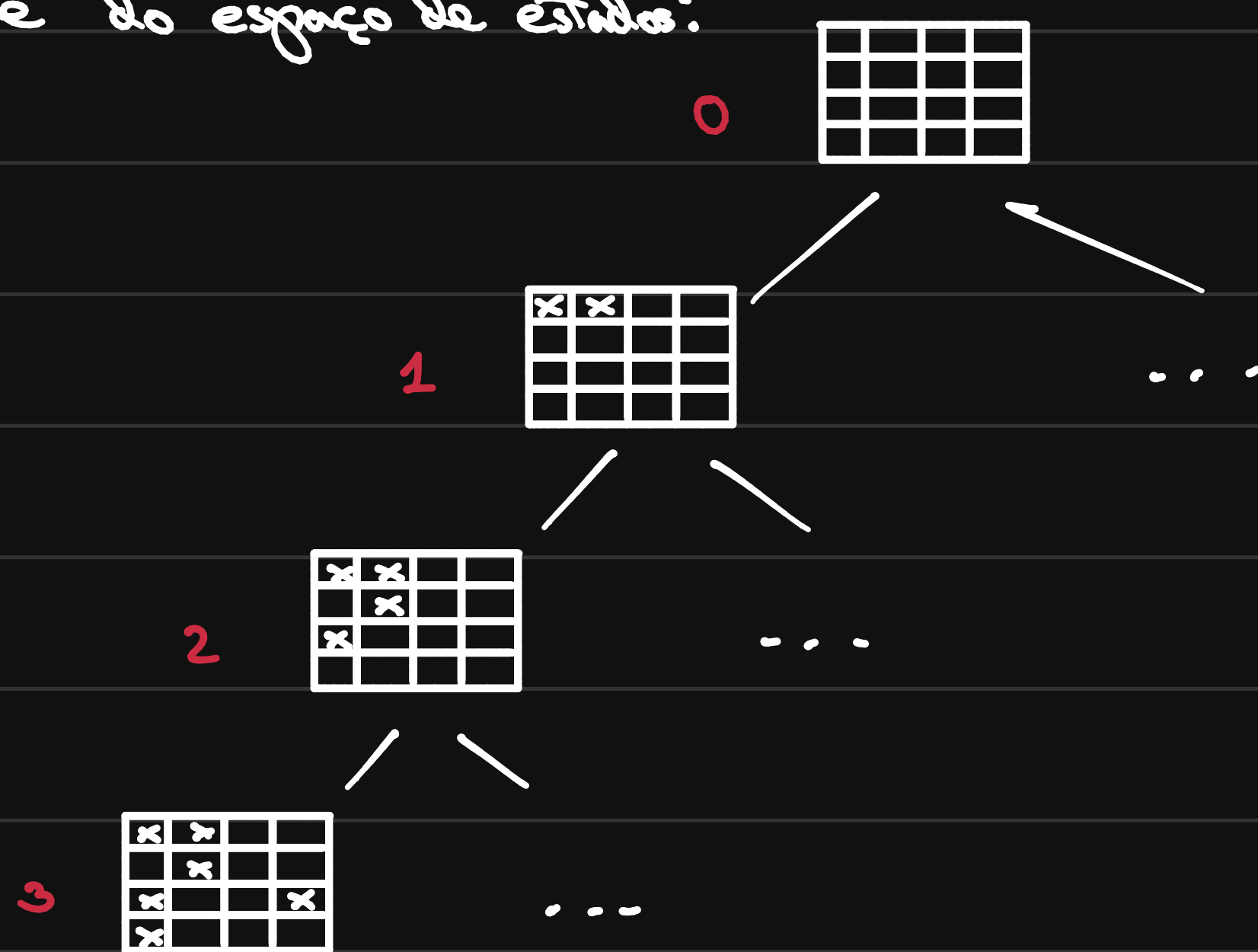
1 = existe X

estado ([ (1,1), (1,2), (2,2), (3,1), (3,4), (4,1), (4,3), (4,4) ]).

ou

Estado = [ [1,1,0,0],  
[0,1,0,0],  
[1,0,0,1],  
[1,0,1,1] ].

Árvore do espaço de estados:



2.  $\text{lim}(L, C) :- \text{between}(1, 4, L),$   
 $\text{between}(1, 4, C).$

$\text{sucessor}(\text{Estado}, \text{EstadoSucessor}) :- \text{lim}(L1, C1),$   
 $\text{lim}(L2, C2),$   
 $(L1 \neq L2 ; C1 \neq C2),$   
 $\neg \text{member}(L1, C1, \text{Estado}),$   
 $\neg \text{member}(L2, C2, \text{Estado}),$   
 $\text{EstadoSucessor} = [(L1, C1), (L2, C2) | \text{Estado}],$   
 $\text{verifica\_restricoes}(\text{EstadoSucessor}).$

3.  $\text{conta\_linhas}(-, [ ], 0).$

$\text{conta\_linhas}(L, [(L, -) | R], N) :- \text{conta\_linhas}(L, R, N1),$   
 $N \text{ is } N1 + 1.$

$\text{conta\_linhas}(L, [ - | R ], N) :- \text{conta\_linhas}(L, R, N).$

$\text{conta\_colunas}(-, [ ], 0).$

$\text{conta\_colunas}(C, [( -, C) | R ], N) :- \text{conta\_colunas}(C, R, N1),$   
 $N \text{ is } N1 + 1.$

$\text{conta\_colunas}(C, [ - | R ], N) :- \text{conta\_colunas}(C, R, N).$



verifica-restricoes(Estado):- forall(between(1,4,L), (conta-linhas(L, Estado, NL), NL =< 4)),  
forall(between(1,4,L), (conta-colunas(L, Estado, NC), NC =< 4)).

4. a) Forward checking: Sim

Verifica antecipadamente se adicionar uma peça torna impossível cumprir as restrições, evitando estados inválidos e cortando caminhos logo no início.

b) Escolher a variável mais restringida: Sim

Escolher a linha/coluna com mais peças limita as opções, acelerando a busca e evitando caminhos maus.

c) Escolher a variável menos restringida: Não

Pode gerar estados inviáveis mais tarde, resultando em maior espaço de pesquisa.